Lab 09 — Compose: orchestrating the stack

Theory — how to replace three ten-line `podman run` commands with one declarative file, start the services in the right order, and keep the same file from the developer's workstation to staging.

Objectives

- Understand what Compose adds over scripts of `podman run`, and what it does not do.
- Read and write a `compose.yaml`: `services`, `volumes`, `networks`, `secrets`, `depends_on`, `healthcheck`.
- Master the `up` / `ps` / `logs` / `exec` / `down` cycle, and what `down` destroys or keeps.
- Handle several environments: `compose.override.yaml`, `-f`, `.env`, `podman compose config`.
- Know what `podman compose` actually runs, and how it differs from Docker Compose.

---

1. The problem: the stack fits in three commands… of ten lines each

In lab 07, the `web → api → db` chain required a `podman network create`, a volume, three `podman run` commands with their `-e`, `-v`, `-p`, `--secret`, in the right order, then the reverse clean-up. Every colleague types those commands slightly differently; nobody knows which version is running; and the day a service must be added, everybody digs through their shell history.

Compose replaces all of that with one file, `compose.yaml`, that **describes** the desired state: the services, their images, their variables, their volumes, their dependencies. One command, `podman compose up -d`, creates whatever is missing; another, `podman compose down`, removes everything cleanly. The file is versioned with the code: the stack becomes reproducible.

> **→ HISTORY**
> In 2014, a small company, Orchard, released **Fig**, a Python tool to describe several Docker containers in a YAML file. Docker bought Orchard the same year; Fig became `docker-compose` (2015). In 2020, Docker published the **Compose Specification**, an open standard, and rewrote the tool in Go: `docker compose` (v2, no hyphen). `podman-compose`, written in Python since 2019, implements that same specification on top of Podman.

2. Anatomy of a `compose.yaml`

```yaml
name: stack                                  # project name (otherwise: the folder)
services:
  db:
    image: docker.io/library/postgres:16-alpine
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
    volumes:
      - pgdata:/var/lib/postgresql/data      # named volume (declared below)
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 5
  api:
    image: localhost/api:${API_TAG:-1.0}     # substituted from .env
    build:
      context: ../05                         # or built from this Dockerfile
    env_file: [api.env]
    depends_on:
      db:
        condition: service_healthy           # waits until db is "healthy"
  web:
    image: docker.io/library/nginx:alpine
    ports: ["18080:80"]
    volumes:
      - ./nginx-proxy.conf:/etc/nginx/conf.d/default.conf:ro   # bind mount
    depends_on:
      api:
        condition: service_healthy
volumes:
  pgdata:
secrets:
  db_password:
    file: ./db_password.txt
```

Every key under `services` matches a `podman run` option you already know: `image` / `build`, `ports` (`-p`), `environment` (`-e`), `env_file` (`--env-file`), `volumes` (`-v`), `secrets` (`--secret`), `restart` (`--restart`), `healthcheck` (`HEALTHCHECK`), `command` / `entrypoint`. Three things are **implicit**: a **network** `<project>_default` is created and every service is attached to it; every service is reachable there by **its name** (the DNS from lab 07); and the containers are called `<project>_<service>_1`.

→ YAML
YAML relies on **indentation** (two spaces, never a tab): `db:` under `services:` is a service, `pgdata:` under `volumes:` is a volume. A value with nothing after the colon (`pgdata:`) is empty: "create it with the default options". Lists are written with `-` or inside square brackets; strings containing `:` (such as `"18080:80"`) go in quotes, otherwise YAML reads a key. `podman compose config` prints the file as the tool understood it: the first reflex when "it does not do what I wrote".

3. The life cycle

Command	Effect	Note
`podman compose up -d`	creates network, volumes, secrets, containers; starts in `depends_on` order	run again, it only recreates what changed
`podman compose ps`	state of the project's services	`podman ps` shows the same containers
`podman compose logs -f api`	logs of one or all services	prefixed with `[api] |`
`podman compose exec db psql …`	command inside a service	`podman exec stack_db_1 …` does the same
`podman compose stop` / `start` / `restart`	without destroying	like `podman stop/start`
`podman compose down`	stops and removes containers, network, pod	**keeps the volumes**
`podman compose down -v`	same, **plus the volumes**	the database disappears
`podman compose build`	builds the services that have `build:`	`up --build` chains both
`podman compose config`	prints the merged and substituted configuration	read it before `up`

The **project name** is the key to everything: it prefixes containers, network and volumes. By default it is the folder name; `name:` in the file (or `-p`) pins it. Running the same file from a renamed folder creates **a second project**, with an empty `other_pgdata` volume: "the database vanished" is the classic symptom.

4. Starting in the right order

`depends_on: [db]` alone guarantees that `db` is **started** before `api` — not that it is ready. PostgreSQL takes two seconds to accept connections; Spring Boot tries to connect immediately and stops with `Connection refused`. Hence the long form: `condition: service_healthy`, which waits until `db`'s **healthcheck** succeeds.

A healthcheck is a command run **inside** the container at regular intervals; its exit code decides: `0` healthy, anything else sick. After `retries` consecutive failures, the state becomes `unhealthy`. It can come from the Dockerfile (`HEALTHCHECK`, lab 10) or from `compose.yaml`, which takes precedence. The command must **exist in the image**: `curl` is not in `eclipse-temurin:21-jre-alpine`, `wget` (BusyBox) is.

→ SPRING BOOT
An ordered start does not let the application off the hook: the database can fall over **later**. A robust service retries (`spring.datasource.hikari.initializationFailTimeout=-1` starts without a database and waits), and `/actuator/health` becomes the API's natural healthcheck — it is what `web` waits for in our stack. In Kubernetes (lab 12), these same checks are called *readiness* and *liveness probes*.

PODMAN
Under rootless, Podman runs healthchecks with **systemd timers**: without a systemd session (a WSL distribution without `systemd=true`), the state stays `starting` forever, and `condition: service_healthy` blocks `up`. Check `systemctl --user status` before blaming Compose. Docker schedules healthchecks inside its daemon.

5. One file, several environments

Three mechanisms combine:

- **`compose.override.yaml`** is merged **automatically** with `compose.yaml`: put there what is true only in development (publish the database on `127.0.0.1:5432`, mount the sources, a `dev` profile).
- **`-f compose.yaml -f compose.prod.yaml`** merges the given files explicitly, in order, and **then ignores the override**: `restart: unless-stopped`, no port for the database, `prod` profile.
- **`.env`** next to the file supplies the variables **substituted into the YAML**: `${API_TAG:-1.0}` is `API_TAG` if set, otherwise `1.0`. `API_TAG=2.0 podman compose config` shows it.

PITFALL
`.env` and `env_file` have **nothing** in common. `.env` is read by Compose to replace the `${…}` in the YAML file; it never enters the containers. `env_file: [api.env]` is passed to the container as `--env-file`. An `APP_MESSAGE=…` placed in `.env` never reaches the API. Second pitfall: `podman compose exec` re-reads the default files to inject the service's environment — if the project was started with `-f`, give the **same** `-f` to `exec` (or use `podman exec`), otherwise you will see the override's variables.

6. What `podman compose` runs

PODMAN
`podman compose` is only a **relay**: it looks for `podman-compose` (Python, `apt install podman-compose`; pick a version ≥ 1.1, through `pipx` if `apt` 's is old) or `docker-compose`, and hands over — hence the banner `>>>> Executing external compose provider`. Visible differences from Docker Compose: containers are named `stack_api_1` (Docker: `stack-api-1`); podman-compose groups the containers in a **pod** `pod_stack` with no infra container (a mere grouping, each service keeps its IP); `down` cannot remove the instances created by `--scale`; and `restart:` is applied by the container, with no daemon to relaunch it after `wsl --shutdown` (lab 03). For a real automatic start: Quadlet (lab 10). The other route, closer to Docker: `systemctl --user enable --now podman.socket` and the real `docker compose` v2 pointed at `DOCKER_HOST=unix://$XDG_RUNTIME_DIR/podman/podman.sock`.

7. In the workplace

- **Compose is the tool of the workstation and of CI**: one `podman compose up -d` gives every developer the full stack in a minute, and CI uses it for integration tests (`up -d`, tests, `down -v`).

- **It suits small deployments**: one VM, one application, `compose.prod.yaml` and `restart: unless-stopped`. It provides neither high availability, nor zero-downtime updates, nor distribution across machines: that is the boundary with Kubernetes (lab 11).
- **Everything you wrote here carries over**: a service becomes a Deployment, a named volume a PersistentVolumeClaim, a healthcheck a *probe*, `depends_on` a *readiness*, `ports` a Service. Compose is the first draft of your manifests.
- **Secrets** stay out of the file: `secrets:` points to a file ignored by Git, or to an engine secret (`external: true`).

Remember

- Compose describes the desired state in a versioned file; `up` creates what is missing, `down` removes everything except volumes.
- A service = a `podman run` translated into YAML; network and DNS are implicit, by service name.
- The project name prefixes everything; `name:` avoids ghost projects and "vanished" volumes.
- `depends_on` alone waits for the start; `condition: service_healthy` waits for a passing healthcheck — with a command present in the image.
- `compose.override.yaml` is automatic, `-f` is explicit and ignores it; `podman compose config` shows the result.
- `.env` substitutes into the YAML, `env_file` enters the container: two different things.
- `podman compose` relays to podman-compose: names with `_`, a grouping pod, no scale-down, `restart` without a daemon.

Vocabulary

Compose: tool and format for describing a multi-container application. — **project**: named set of resources created from a Compose file. — **service**: one container (or several instances) described under `services:`. — **`depends_on`**: start order, with or without a condition. — **healthcheck**: periodic command that decides the `healthy` / `unhealthy` state. — **override**: file merged on top of `compose.yaml`. — **substitution**: replacement of `${VAR}` from `.env` or the shell. — **`podman compose config`**: display of the merged configuration. — **podman-compose**: Python implementation of the Compose Specification for Podman. — **project pod**: `pod_<project>` grouping created by podman-compose.